SecureCheck: A Python-SQL Digital Ledger for Police Post Logs

Visualizing Traffic Stop Patterns and Insights

Contents

01 Project Overview

02 Data Preparation & Preprocessing

03 Database Design & Integration

04 Dashboard & Insights

05 Conclusion & Future Work

Project Overview

Scope, Objectives, and Context at a Glance

01

Problem Statement & Objectives

Step1

Real-world problem & project goal

Address need for secure, data-driven analysis of police traffic stops by building an end-to-end pipeline from preprocessing to interactive dashboard for evidence-based decision-making.

Step2

Deliverables, tools, and key focus

Deliver cleaned dataset, SQL database, Streamlit dashboard using VS Code, Python, MySQL, PyMySQL, emphasizing data quality, security, usability, and actionable insights for stakeholders.

End-to-End Project Workflow

Data Pipeline & Processing Flow

Raw traffic stop data is ingested into VScode for cleaning, feature engineering, and quality checks, then stored as structured tables in MySQL police_secure_check1 and secure_check schemas.

Dashboard, Feedback Loop & Modularity

Streamlit dashboard connects to MySQL via PyMySQL, visualizing key metrics; user insights drive iterative preprocessing and model refinement, with each workflow component independently modular and improvable.

Data Preparation & Preprocessing

Transforming Raw Data into Reliable Analytical Assets

02

Data Understanding & Cleaning Steps

01

Preprocessing Workflow Overview

Load raw dataset in VS code, inspect structure, review key columns like stop_date, stop_time, search_type, then scan for missing values, type issues, and inconsistent entries.

02

Cleaning Execution & Environment

Apply defined cleaning rules, correct data types, handle missingness and inconsistencies, re-verify dataset integrity, and document that all preprocessing occurred within VS Code's integrated Jupyter environment.

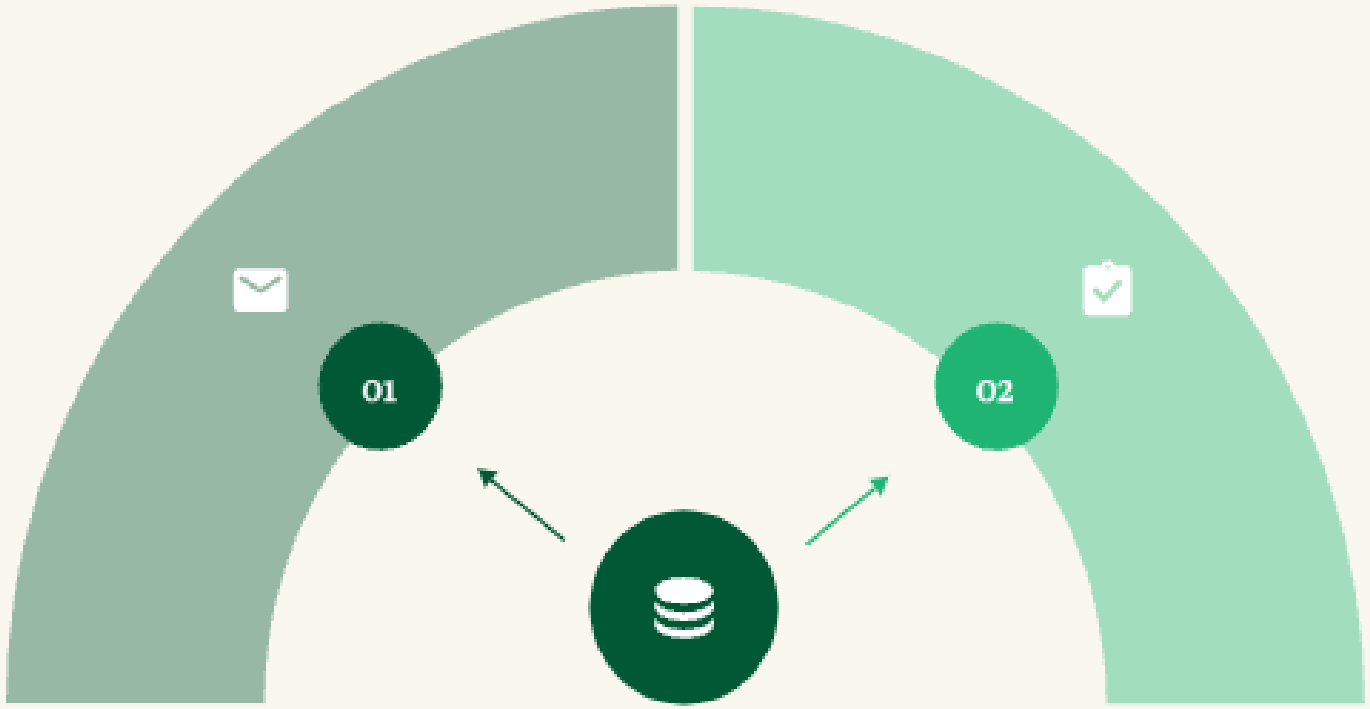
Handling Missing Values & Data Types

Overall strategy for clean, reliable data

Focus on categorical nulls only, imputing with mode to preserve dominant patterns, minimize information loss, and keep the dataset stable for downstream time-based analytics.

Specific treatments and validation checks

Converted date and time fields from string to proper datetime and time types, then confirmed no critical nulls remained and all types aligned with SQL schema and dashboard requirements.



Database Design & Integration

Building Seamless, Scalable, and Connected Data Systems

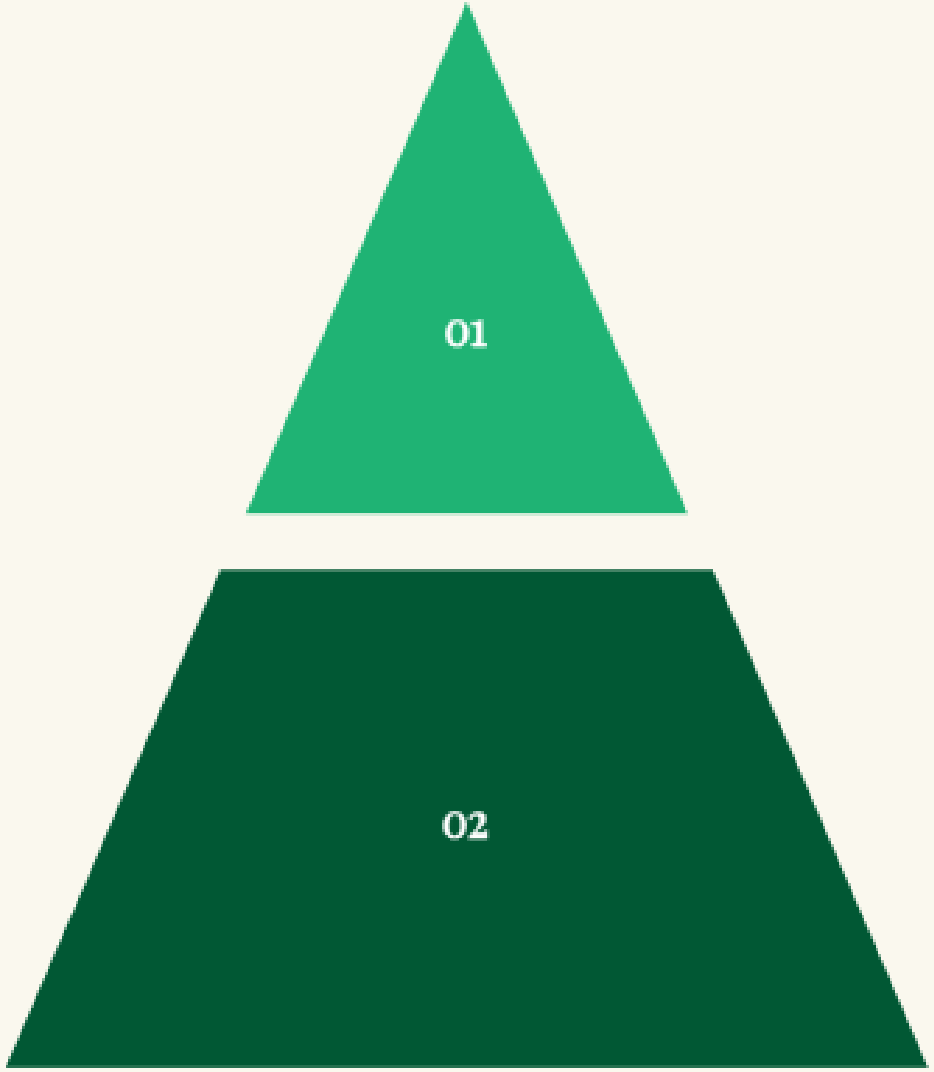
03

MySQL Database & Table Structure

01

Hierarchical structure: database to key columns

Project-level MySQL database contains core fact table for traffic stops, organizing all records centrally. Table holds each stop event as a row for consistent, scalable storage.



02

Key columns, primary key, and query performance

Key columns like stop_id, driver_id, stop_date, stop_time, search_type, violation, outcome enable filtering. Proper primary key selection and indexing ensure fast dashboard queries.

Data Insertion & Validation Flow

01

Database setup and data loading

Connect from Python to MySQL, create project database and tables, define columns and data types, then insert cleaned records via bulk insert or iterative loops as needed.

02

Verification and basic security checks

Run sample SELECT queries to confirm row counts and integrity; highlight avoiding hard-coded credentials, using environment variables, and restricting database permissions in demonstrations.

Dashboard & Insights

Transforming Data into Clear, Actionable Intelligence

04

Streamlit–MySQL Architecture

End-to-End Data Flow

Users choose filters in Streamlit; PyMySQL sends SQL queries to MySQL, retrieves results, then displays interactive tables and visual summaries back in the Streamlit interface.

Analytics and Separation of Concerns

Python computes predictions and insights, rendering charts and metrics in Streamlit; Streamlit acts as frontend, MySQL as backend, with PyMySQL bridging communication cleanly.

Key Dashboard Features & Insights

Interactive Filters & Visual Exploration

Users filter stops by date, time, violation, and search type, then explore distributions over time, search versus no-search proportions, and aggregated outcomes such as warnings, citations, or arrests.

Predictive Component & Decision Support

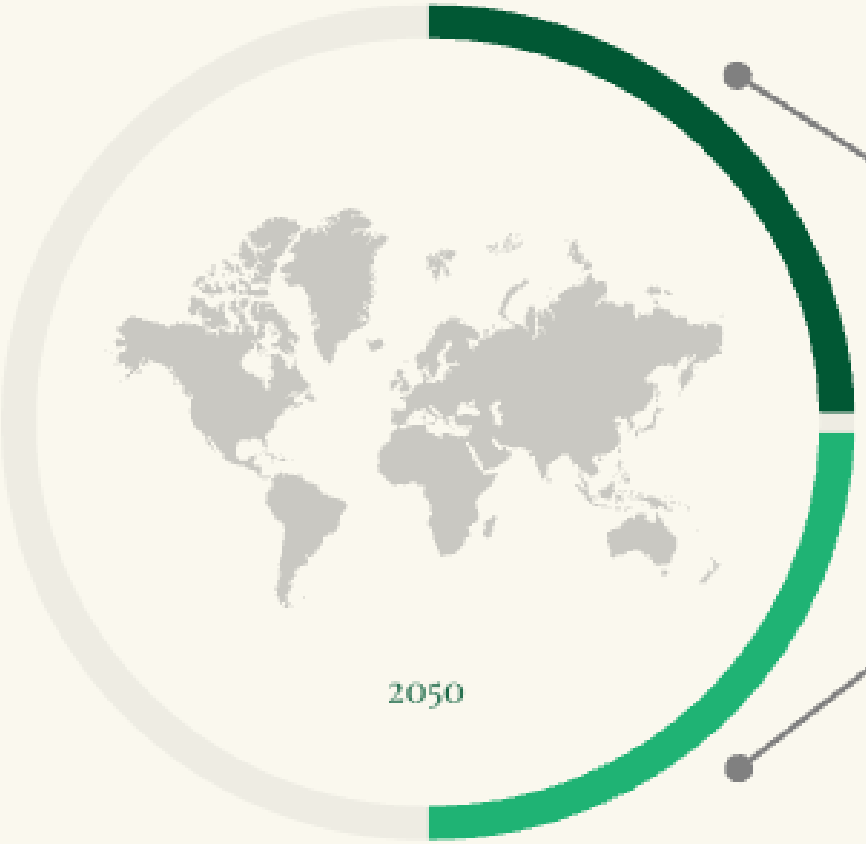
A simple rule-based or machine-learning model estimates probability of searches given stop attributes, revealing patterns and supporting evidence-based policy, training, and operational decision-making.

Conclusion & Future Work

Key Insights and Directions for Future Research

05

Project Outcomes & Learnings



Key project outcomes

Delivered full pipeline from preprocessing to SQL storage and interactive dashboard. Demonstrated robust data cleaning, type handling, database integration, and web app deployment in a cohesive, reproducible workflow.

1

Technical skills and personal growth

Strengthened Python, SQL, and Streamlit stack; improved database design and integration. Gained deeper appreciation for clean data and learned to architect complete, maintainable pipelines over isolated scripts.

2

Limitations & Future Enhancements

Current Limitations

Dashboard relies on a limited feature set and possibly small dataset, with basic prediction logic and incomplete security and authentication hardening for production-ready usage.

Future Work & Deployment

Expand data dimensions, strengthen model training and validation with metrics, add role-based access, and deploy Streamlit app with cloud database for secure, real-time access.

Thanks

Gratitude, Appreciation, and the Power of Thanks

Presenter: Keerthana G.C.